

ChargeWise AI

FIAP x GoodWe Brasil | Prompt and Artificial Intelligence | Sprint 03 | 21/09/2026

1. Resumo da evolução e escopo

Assistente para recarga compartilhada em condomínios. Moradores consultam disponibilidade, estado, potência cadastrada, consumo e cobrança; a administração é público secundário. A aplicação Python roda localmente, com inferência local ou cloud. Todos os dados operacionais são simulados.

A Sprint 2 não foi entregue. O “antes” disponível é uma base criada posteriormente e preservada em evals/baseline. Esta origem é declarada para não apresentar a implementação atual como uma entrega histórica. O projeto evoluiu para cadeia LCEL, memória por sessão, contrato Pydantic v2 e validação de evidências.

Base posterior / referência manual	Sprint 03 — versão final
Histórico simples por turnos	Memória por tokens, resumo extrativo e isolamento
Contrato inicial e verificações básicas	Estado, potência, consumo, valor e fontes validados
Integração inicial / demonstração por regras	ChatPromptTemplate → ChatOllama → captura → parser
Cenários iniciais	29 casos comuns, tentativas brutas e métricas

O roteador e a resposta operacional exibida são controlados por regras. O modelo preenche campos que precisam corresponder ao contexto; o texto bruto é preservado para análise. Essa escolha reduz afirmações não fundamentadas, mas limita a geração aberta e exige separar avaliação do modelo de avaliação das regras.

Equipe

Integrante	RM
Arthur de Oliveira	568986
Miguel Piedade	572445
Rafael Zani	569033

Turma: 1CCPI. Repositório informado pela equipe: github.com/Zanizada/chargewise-ai-3.0; acesso público não verificado nesta preparação. A divisão de trabalho está na última página.

2. Refatoração, decisões e problemas

Fluxo: pergunta → escopo/moderação → contexto e orçamento → cadeia LCEL → parser Pydantic → validação de evidências → renderização controlada → gravação do histórico. Respostas rejeitadas não se tornam contexto de novos turnos.

RunnableWithMessageHistory envolve a execução validada. ConversationTokenBufferMemory limita mensagens; a implementação remove pares antigos e preserva resumo extrativo de intenção, entidade e fontes. Valores antigos não entram no resumo. Tiktoken estima o orçamento; contadores do serviço são registrados separadamente.

Os parâmetros comuns foram temperature=0,2, top_p=0,9, num_predict=2048, timeout=120 s, até um retry, memória=1500, resumo=200 e contexto=8192 tokens. As classes exigidas pela disciplina foram mantidas apesar dos avisos de descontinuação.

Problema identificado	Decisão / resultado
JSON válido com fonte ou valor divergente	Conferir campos e fontes contra o contexto antes de salvar. Falhas permanecem registradas, sem substituir por demo.
Repetição de mensagens em retries	Gravar somente o turno validado e restaurar o estado em falha.
Histórico pode exceder o orçamento	Poda de pares e resumo extrativo limitado; contexto financeiro relido das fontes.
Cloud configurado como localhost	Endereço ajustado para ollama.com, preservando a chave local.
Caractere Unicode não exibido pelo terminal	Executar Python com -X utf8; consulta real passou após esse ajuste.

Guardrails recusam padrões de injeção e encaminham aconselhamento restrito. Fontes e números são conferidos. Não há garantia universal contra ataques, autenticação, persistência, cobrança real, reserva ou comando em equipamento.

A configuração utiliza .env local ignorado pelo Git. O repositório deve conter apenas .env.example sem chave. Os testes de componentes e HTTP simulado validam integração, mas não substituem os experimentos reais descritos a seguir.

3. Resultados reais — dois modelos

Qwen3:8b local (Python 3.13.2) e gpt-oss:120b no Ollama Cloud (Python 3.12.10). Uma repetição por modelo, 29 casos cada. Em cada execução, 21 turnos chamaram o modelo e oito foram resolvidos sem inferência.

Métrica	Qwen local	GPT-OSS cloud
Casos avaliados	29	29
Checagens básicas aprovadas	27/29 (93,10%)	26/29 (89,66%)
Falhas de execução/validação	2	1
Turnos com chamada ao modelo	21	21
Tentativas, incluindo retries	29	23
Latência média — todos os turnos (s)	7,23	1,24
Latência média — turnos com modelo (s)	9,99	1,72
Tokens de entrada/turno com modelo	3001,10	2090,52
Tokens de saída/turno com modelo	632,24	389,86
Schema válido por tentativa	28/29 (96,55%)	23/23 (100%)
Schema válido na primeira tentativa	20/21 (95,24%)	21/21 (100%)
Acurácia dos campos esperados	12/12 (100%)	9/12 (75%)
Nota qualitativa humana (0-2)	Não avaliada	Não avaliada

Médias de tokens: soma de tentativas por turno, com cobertura de 21 turnos em ambos. Latência inclui aplicação e retries. Acurácia de campos usa 12 campos esperados; schema usa tentativas. Essas métricas não equivalem a uma nota qualitativa humana.

Qwen reprovou M03 e E06, ambos por validação. GPT-OSS reprovou F01 por validação; F02/F03 retornaram respostas incompatíveis com as expectativas na sequência de potência. O primeiro turno rejeitado prejudica a continuidade. Não se conclui superioridade geral de um modelo.

Comparação descritiva de duas execuções, com uma repetição por modelo. Os hashes dos casos e contextos coincidem. O hash do arquivo system_prompt_v3.md difere; portanto, a etiqueta v3 não identifica o mesmo conteúdo nos dois testes. Qwen foi executado localmente e GPT-OSS no Ollama Cloud. Não é possível atribuir diferenças exclusivamente ao modelo, à arquitetura ou ao prompt.

4. Comparativo antes/depois e fechamento

Métrica obrigatória	Sprints 1/2 — referência manual posterior	Sprint 03 — LCEL real
Qualidade das respostas (nota no eval)	N/D — sem avaliação real/humana	N/D — notas humanas pendentes
Tokens de entrada por turno com modelo	N/D — sem execução manual real	Qwen 3001,10; GPT-OSS 2090,52
Latência média, todos os turnos	N/D — sem execução manual real	Qwen 7,23 s; GPT-OSS 1,24 s
Acurácia de campos da saída estruturada	N/D — sem execução manual real	Qwen 100%; GPT-OSS 75% (12 campos)

Evidência demo separada: base 21/29 checagens (72,41%) e final 29/29 (100%); campos 50% e 100%. Esses números verificam regras determinísticas, não demonstram ganho de um LLM. A tabela acima mantém N/D onde não existe medição real.

O projeto dispõe de prompts v1/v2/v3. Não há experimento real v2/v3 do mesmo modelo neste conjunto; portanto, não se declara ganho medido de prompt. Os dois arquivos chamados v3 diferem em hash. Os hashes de casos e contextos coincidem; o manifesto em docs/comparativo_execucoes.json mantém a origem de cada resultado.

Evidências e reprodução

Os JSONs originais estão em evals/runs/ e foram preservados byte a byte. O relatório de modelos detalha parâmetros, denominadores e falhas. Para reproduzir GPT-OSS: configurar cloud e executar `python -X utf8 -m evals.executar --modo real --ids lcel_120_v3 --repeticoes 1`. Para Qwen local, usar `lcel_qwen_v3` e seu endpoint. O prompt histórico do Qwen precisa ser recuperado para repetição exata.

Pendências acadêmicas explícitas

Revisar notas, justificativas e avaliador nos 58 registros; executar a baseline manual real e o par v2/v3 controlado; consolidar `sprint3_results.json`; confirmar acesso público ao repositório. Não se substitui essa revisão por porcentagens automáticas. O relatório reúne as evidências disponíveis, sem declarar a rubrica integralmente cumprida.

Integrante	Tarefa informada pela equipe
Arthur de Oliveira	Prompts e Evals
Miguel Piedade	src, LangChain, validação e README
Rafael Zani	Guardrails, organização de branch, chain e schemas

Uso de IA apoiou código, execução e documentação. A equipe deve revisar a entrega e confirmar a autoria das contribuições. Escopo baseado no compilado de instruções das Sprints 1, 2 e 3 fornecido pelo grupo.